

给我讲讲LSP (Language Server Protocol), 我刚刚看到有个
textDocument/implementation
似乎是LSP的一个接口

展开 ∨

可以把 **LSP (Language Server Protocol)** 想成一种“编辑器 和 编程语言工具之间的通用语言”。你写代码的编辑器（比如 VS Code）和真正懂某种语言的“智能分析器”（语言服务器）之间，不再是各搞各的，而是通过一套标准协议交流。

一、先从直觉理解：为什么需要 LSP?

在 LSP 出现之前：

- 每个编辑器都要**单独支持每种语言**
- 每种语言也要**为每个编辑器写插件**

👉 结果是：组合爆炸 🤯

比如：

- Java × VS Code
- Java × IntelliJ
- Python × VS Code
- Python × Vim

都要分别实现

二、LSP 的核心思想

LSP 把问题拆成两部分：

1 编辑器 (Client)

负责：

- 展示代码
- 接收用户输入
- 显示跳转 / 提示 / 报错

2 语言服务器 (Server)

负责：

- 解析代码 (AST 就在这里派上用场)
- 语义分析
- 提供“智能功能”

两者通过 JSON-RPC 通信 (本质是发消息)：

编辑器：我这里有个文件

语言服务器：收到，我解析一下

编辑器：光标在这里，给点提示？

语言服务器：这是个函数，参数是xxx

三、你提到的 AST 在哪用？

你已经学过 AST，这刚好是 LSP 的核心基础 📌

语言服务器内部大概流程：

源码 → Token → AST → 语义分析 → 符号表

然后基于这些能力实现：

功能	本质
自动补全	查符号表
跳转定义	找变量定义位置
查引用	反向查找
类型提示	类型推导
implementation	找实现

四、重点：textDocument/implementation 是啥？

这是 LSP 的一个请求接口。

👉 它的作用：

从“接口/抽象定义”跳到“具体实现”

举个例子 (Java / TS 都类似)

```
interface Animal {
  speak(): void;
}

class Dog implements Animal {
  speak() { console.log("woof"); }
}
```

当你光标在:

```
Animal speak
```

调用:

```
textDocument/implementation
```

👉 返回:

```
Dog.speak
```

和 definition 的区别

接口	含义
textDocument/definition	去“声明的位置”
textDocument/implementation	去“具体实现”

再举个更现实的例子

```
function process(a: Animal) {  
  a.speak();  
}
```

光标在 `speak` :

- definition → interface `Animal.speak`
- implementation → `Dog.speak` / `Cat.speak` / etc

👉 也就是说:

implementation = 找“谁真正干活”

五、LSP 的核心接口长什么样？

大致是这样的 JSON:

```
{  
  "method": "textDocument/implementation",  
  "params": {  
    "textDocument": { "uri": "file:///a.ts" },  
    "position": { "line": 10, "character": 5 }  
  }  
}
```

返回:

```
[  
  {  
    "uri": "file:///dog.ts",  
    "range": { ... }  
  }  
]
```

六、你可以这样理解 LSP (非常关键)

LSP = 把“编译器的中间能力”变成“编辑器的实时服务”

你在编译原理里学的:

- AST
- 符号表

- 类型系统

👉 全都被“在线化”了

七、为什么这个设计很强？

因为：

- 编辑器不用懂语言
- 语言只要实现一次 LSP server
- 所有编辑器都能用

这也是为什么：

- VS Code 能支持几百种语言
 - 很多语言只提供一个 language server 就够了
-

八、你现在这个阶段该怎么理解最合适？

用一句话总结你当前认知层级：

LSP = “基于 AST 的语义能力，通过 RPC 暴露给编辑器”

如果你想更深入（我可以带你继续）

可以继续聊这些方向：

- ↳ LSP server 内部如何维护 AST（增量更新）
- ↳ hover / completion 是怎么实现的
- ↳ implementation 在动态语言（Python）里的难点
- ↳ 怎么自己写一个最简 LSP server（很适合你现在）

你想往“原理深挖”还是“动手实现一个”？

📄 🍷 🗨️ ↻ ...

卧槽 🤔 🤔 🤔 设计出LSP的人真是个天才 😊 😊 😊

那PSI (Program Structure Interface) 呢？也就是说PSI是用LSP做的吗

先给你泼点冷水：PSI 不是用 LSP 做的，甚至它们是两种完全不同时代/思路的东西。

一句话结论

PSI $\neq$ LSP 的实现

PSI = IDE 内部结构

LSP = IDE 和语言工具之间的通信协议

一、PSI 是啥？

PSI 来自 JetBrains 体系（比如 IntelliJ IDEA）

它的全称是：

Program Structure Interface

你可以把它理解成：

👉 “比 AST 更高级的一层代码结构模型”

二、先对比 AST vs PSI（这个很关键）

你现在的认知在 AST，对吧，我们直接升级一下：

AST（你学过的）

```
a + b
```

AST 可能是：

```
BinaryExpression
├─ Identifier(a)
└─ Identifier(b)
```

👉 特点：

- 贴近语法
- 偏“编译器内部结构”
- 不关心编辑体验

PSI (IDE 用的)

同样代码，在 PSI 里：

- a 是一个变量引用 (可以 resolve)
- b 是另一个变量
- a + b 是一个表达式节点
- 可以直接：
 - 跳转定义
 - 找引用
 - 重构

👉 PSI 是：

带语义 + 可操作 + 可修改 的 AST 超集

三、PSI 到底多强？

PSI 的设计目标是：

👉 让 IDE 可以“像人一样理解代码”

所以它支持：

- resolve (变量指向哪里)
- rename (重命名自动改所有引用)
- find usages (查引用)
- refactor (提取函数等)

这些能力都直接挂在节点上：

```
psiElement.resolve()
psiElement.getReferences()
psiElement.rename()
```

👉 这已经不是“语法树”，而是“代码对象模型”了

四、那 LSP 在干嘛？

LSP 是另一种思路：

👉 不在 IDE 里做这些事，而是交给外部 server

对比一下：

	PSI 模式	LSP 模式
计算位置	IDE 内部	外部 server
AST	IDE 自己维护	server 维护
交互方式	API 调用	JSON-RPC
性能	极快（内存内）	有通信开销
扩展性	差	极强

五、核心区别（本质）

PSI

👉 “我自己就是编译器 + IDE，一体化”

(典型：IntelliJ IDEA)

LSP

👉 “我只是 UI，编译器你去外面跑”

(典型：Visual Studio Code)

六、一个非常形象的比喻

PSI (JetBrains)

像一台“全能战舰” 🚢
引擎、武器、雷达全在内部

LSP (VS Code)

像“无人机 + 云端控制” 🚁
本地很轻，算力在外面

七、为什么 JetBrains 不用 LSP?

因为:

- PSI 已经**更强 + 更深度集成**
- 不只是“查询”，还支持：
 - 修改代码结构
 - 精确重构
 - 复杂语义操作

而 LSP:

👉 更偏“查询接口”，不是“操作模型”

八、但是！现在有融合趋势

现实世界不是非黑即白:

- JetBrains 也开始支持 LSP (某些语言)
- VS Code 也在增强本地分析能力

👉 趋势是:

PSI (强模型) + LSP (通用协议) 融合

九、你现在这个阶段最该抓住的理解

你可以这样建立 mental model:

你学的编译原理:

源码 → AST → 语义分析



PSI:

AST → PSI (可操作语义模型)



LSP:



十、给你一个“顿悟级总结”

PSI 是“数据结构的革命”

LSP 是“系统架构的革命”

如果你想更进一步（强烈推荐）

你现在正好在一个很好的学习点，可以继续深挖：

1. PSI 是怎么做“增量更新”的（比 AST 难很多）
2. resolve 是怎么实现的（符号表进阶版）
3. 自己写一个 mini LSP server（很适合你）
4. 对比 TypeScript language server vs IntelliJ TS 支持

你要不要我带你做一个：

👉 “从 AST → 实现一个最简 LSP（支持 definition + implementation）”

这个项目会直接让你理解一整套体系。



😳😳😳 你阅读一下当前项目里的另一个对话，你就知道我是想做一个coding agent了
那这样的话，其实我自己做是需要用LSP来实现一个PSI吗，还是怎么说？因为我觉得让Agent通过PSI来改代码才比较稳

你这个直觉是对的，但你现在的表述有点“混层了”，我帮你掰直一下：

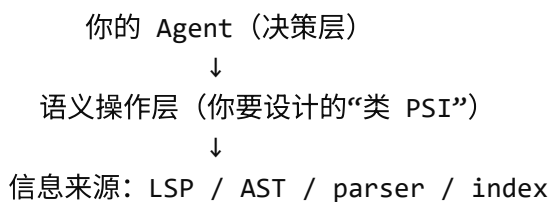
❌ “用 LSP 实现 PSI”

✅ “用 LSP + 自己的中间模型，做一个『类 PSI』的能力层”

一、先把关键关系摆正（非常重要）

你要做的是 coding agent，对吧？

那体系其实是这样：



👉 关键点：

LSP 是“信息来源”，不是“操作模型”

二、为什么不能直接把 LSP 当 PSI 用？

因为 LSP 天生有几个限制：

1 它是“查询式”的

```
textDocument/definition
textDocument/references
textDocument/hover
```

👉 都是：问 → 返回结果

但你要的是：

```
renameSymbol()
extractFunction()
moveFile()
```

👉 这是“操作”

2 它没有“结构对象”

LSP 返回的是：

```
{
  "uri": "...",
  "range": { ... }
}
```

👉 只是“坐标”，不是“对象”

而 PSI 是：

```
functionNode.rename("newName")
```

👉 是可以操作的对象

3 它不保证一致性（对 agent 很关键）

Agent 需要：

- 知道修改会影响哪里
- 能 rollback
- 能做多步推理

LSP：

✗ 没有事务 / 没有全局视图 / 没有操作语义

三、那你应该怎么设计？（重点来了）

你真正要做的是：

👉 在 LSP 之上，构建一个“Agent-Friendly PSI 层”

四、推荐架构（非常适合你现在阶段）

💡 分三层：

```
[ Layer 1 ] LSP / Tree-sitter / Parser
            (数据来源)
```

```
[ Layer 2 ] Code Graph / Symbol Graph
```



(你自己建的)

[Layer 3] Agent Actions (类 PSI API)

五、每一层干嘛？

Layer 1: LSP (你不用 reinvent)

用来获取：

- definition
- references
- implementation
- hover (类型信息)

Layer 2: 你要重点做的 (核心壁垒)

构建一个：

👉 Symbol Graph / Code Graph

比如：

```
Function: processOrder
├─ calls → validateUser
├─ calls → chargeCard
└─ calledBy → handleRequest
```

以及：

```
Symbol: UserService
├─ implementations → StripeUserService
└─ references → 18 places
```

👉 这一步 = “弱 PSI”

Layer 3: 给 Agent 用的 API (最关键)

你要给 Agent 的不是 LSP:

而是：

```
renameSymbol("UserService", "AccountService")

extractFunction({
  range: ...
})

findImpact("change this function")
```

👉 这就是：

PSI-like 操作接口（但你自己定义）

六、一个现实可行的 MVP 路线（建议你这么做）

✅ V1（强烈建议）

不要一开始就做完整 PSI

而是：

👉 **LSP + 文本 patch + 简单语义校验**

流程：

1. 用 LSP 找 definition / references
2. Agent 生成 patch (diff)
3. 应用 patch
4. 再用 LSP 验证（有没有报错）



👉 这是现在很多 agent（包括 Cursor 类工具）的做法

✅ V2（进阶）

引入：

- symbol graph
- dependency graph

👉 开始能做：

- 影响分析
- 更稳的 refactor

🚀 V3 (你说的“PSI 感觉”)

实现：

```
symbol.rename()  
file.move()  
class.extract()
```

👉 这时候你已经：

做出了一个“轻量版 PSI for Agent”

七、关键认知（你必须抓住）

！ PSI 的核心不是“结构”，而是“可安全操作代码”

而 LSP：

！ 只解决了“我能看懂代码”

你要做的 agent：

！ 必须解决“我能安全改代码”

八、一个很现实的建议（可能有点逆耳但很重要）

如果你现在就想：

“我要实现一个完整 PSI 给 agent 用”

👉 我建议你先别这么做

因为：

- 太重（JetBrains 做了 20 年）
- ROI 不高（前期）

更聪明的路径是：

✅ “用 LSP + patch + 校验”先跑通闭环

九、给你一个非常贴你项目的总结

你现在的 voyager 项目，本质可以这样定义：

👉 “一个基于 LSP 的、逐步进化为 PSI-like 的 coding agent”

如果你愿意，我们可以下一步直接落地

我可以👉 帮你设计：

👉 **voyager V1 技术架构（非常具体）**

包括：

- TypeScript 怎么接 LSP
- Agent API 怎么设计
- patch 系统怎么做
- 如何做“安全修改”

甚至可以直接给你：

✅ 项目目录结构 + 核心接口定义（可直接开干）

你要不要直接进“动手设计 V1”？

📄 🍷 🗨️ ↻️ ...

我觉得你说的很对，应该“用 LSP + 自己的中间模型，做一个『类 PSI』的能力层”

我希望你把这一部分给我展开详细说说，我应该怎么做
我的理解是

1，要有一个 LSP Client，去获取 AST 信息

展开 ▼

你的这三点方向是对的，但我先帮你“校准一下模型”，避免你后面走弯路：

✗ “LSP client 获取 AST”

✓ LSP client 获取的是“语义查询结果”，不是 AST 本体

AST 基本不会通过 LSP 暴露（绝大多数 server 不提供），所以你不能把 LSP 当“AST provider”。

一、先给你一个更靠谱的总体架构（适合做 agent）

你要做的其实是：



👉 核心结论：

LSP ≠ 数据源唯一来源

你需要：LSP + 文本 + (可选) AST parser

二、逐条点评你的理解（+升级版）

你说的 **1**：LSP Client 获取 AST

👉 问题：

- LSP 不提供 AST
- 只提供：
 - definition
 - references
 - symbols
 - diagnostics

👉 正确姿势：

你要拿的是这些：

```
definition(symbol)
references(symbol)
documentSymbol(file)
workspaceSymbol(query)
```

👉 用这些拼出“结构认知”

你说的 2：patch / diff 修改代码

👉 这个是 100% 正确，而且是核心能力

但我要帮你升级一下：

！不只是 patch，而是“结构对齐的 patch”

推荐你做三层 patch 能力

● Level 1: Text Patch (MVP)

```
- const a = 1;
+ const a = 2;
```

👉 简单直接 (V1 就用这个)

● Level 2: Range-aware Patch

```
applyEdit({
  uri,
  range: { start, end },
  newText
})
```

👉 和 LSP 的 range 对齐 (关键!)

● Level 3: Semantic Patch (未来)

```
renameSymbol("UserService", "AccountService")
```

👉 内部做：

- 找 definition
- 找 references
- 批量 patch

你说的 3：暴露为 Agent tools

👉 这个完全正确，而且是你系统的“灵魂”

但重点是：

❗ 不要暴露 LSP API，要暴露“语义动作”

三、你真正要实现的核心：Semantic Layer（类 PSI）

这一层我帮你拆细一点（这是壁垒）

1 Symbol Model（最重要）

你需要一个统一结构：

```
type Symbol = {  
  id: string  
  name: string  
  kind: "function" | "class" | "variable"  
  location: Location  
  references?: Location[]  
  implementation?: Location[]  
}
```

👉 数据来源：

- LSP + 你缓存

2 Code Graph（强烈建议做）

构建关系：

```
FunctionA → calls → FunctionB  
ClassA → implements → InterfaceB
```

👉 用来做：

- 影响分析
- agent 推理

3 Document Model (必须有)

你要有：

```
type Document = {  
  uri: string  
  content: string  
  version: number  
}
```

👉 因为：

LSP 是“无状态-ish”，但你必须是“有状态”的

四、核心能力设计（给你一套可以直接实现的 API）

这是重点，我给你一套 V1 就能用的接口

🔧 查询类（读）

```
getDefinition(symbol | position)  
getReferences(symbol | position)  
searchSymbol(query)  
getDocumentSymbols(file)
```

修改类 (写)

1. applyEdit (基础)

```
applyEdit(edit: {  
  uri: string  
  range: Range  
  newText: string  
})
```

2. applyWorkspaceEdit (批量)

```
applyWorkspaceEdit(edits: Edit[])
```

3. renameSymbol (关键能力)

```
renameSymbol({  
  position,  
  newName  
})
```

内部:

1. definition
2. references
3. 生成 edits
4. applyWorkspaceEdit



4. safeEdit (非常建议)

```
safeEdit(edit) {  
  apply  
  runDiagnostics()  
}
```

```
if (error) rollback  
}
```

👉 这是 agent 能稳定的关键

五、业界是怎么做的？

我给你讲真实情况（不是理想化的）

🧠 1. Cursor / Copilot 类

- LSP ✓
- patch ✓
- AST ✗（很少直接用）
- PSI ✗

👉 模式：

text patch + LSP 校验

🧠 2. Sourcegraph

- heavy index（类似 code graph）
- 跨 repo 分析

👉 更偏：

Graph + search + intelligence

🧠 3. JetBrains

- PSI ✓（超重）
- 自己的 parser ✓

👉 但：

✗ 不适合你现在做 agent（太重）

六、一个非常关键但很多人忽略的点

! Agent 最大的问题不是“改代码”，而是“改坏代码”

所以你必须要有：

Safety System (强烈推荐 V1 就做)

1 diagnostics 校验

```
afterEdit → LSP diagnostics
```

2 rollback

```
if (had) revert
```

3 diff review (可选)

让 agent 自己检查：

```
"Does this change break anything?"
```



七、给你一个最现实的 V1 方案（我强烈建议你这么做）

直接可落地：

技术栈

- TypeScript
- LSP client (用现成库)
- 简单内存 document store

能力

✓ 读:

- definition
- references

✓ 写:

- applyEdit
- renameSymbol (自己拼)

✓ 安全:

- diagnostics + rollback

✗ 不做 (现在)

- AST parser (先别碰)
- 完整 PSI
- 复杂 graph

八、最后给你一个“路线总结”

V1: LSP + Patch + Diagnostics
V2: + Symbol Cache
V3: + Code Graph
V4: + PSI-like API



九、我给你的一个直接判断标准 (很重要)

当你设计一个能力时, 问自己:

👉 “这个是 LSP 能直接做的, 还是我要抽象成更高层?”

如果答案是:

- “只是查信息” → 用 LSP
- “涉及修改” → 必须进 Semantic Layer

如果你下一步想继续, 我可以直接帮你:

👉 ↪ 设计 **voyager V1 的 TypeScript 接口定义 + 目录结构 (可直接开写)**

甚至可以↪ 一步步带你把 `renameSymbol` 实现出来（这是最关键的一步）。



我还是偏好Python
1是现在主流的Agent框架都是用的Python，包括但不限于
`openai-agents-python`, `google adk`, `trae-agent`
2是之前讨论过，`adk`似乎比较合适
你看一下呢

你这个选择是合理的，但我得帮你把一个潜在坑提前指出来：

⚠️ **“Agent 用 Python” ≠ “整个系统都该用 Python”**

你现在要做的不是“写一个 agent”，而是：

👉 **一个带代码语义能力的 agent system（带 LSP / 编辑能力）**

这就引出一个关键设计问题：

一、核心判断：Python 适合你哪一层？

我们把系统再拆一次：

[Agent / Orchestration] ← Python（强）
[Semantic Layer] ← ⚠️ 关键选择
[LSP Client / Editor] ← TypeScript（更成熟）



二、直接说结论（基于业界 + 可落地性）

✅ **推荐方案：Python + TypeScript 混合架构**

而不是全 Python

三、为什么不能 All in Python？

不是不能，而是你会踩这些坑：

1 LSP 生态现实

LSP 的“主战场”其实在 JS/TS:

- VS Code extension API
- language servers (很多是 Node 写的)
- LSP client tooling

👉 Python 也有, 但:

! 不够成熟 / 不够贴编辑器生态

2 编辑能力 (你很快会需要)

你迟早会想做:

- 文件操作
- workspace 管理
- 实时编辑同步

这些在:

- Visual Studio Code 生态 (TS)

👉 是“原生能力”

3 性能 & 交互

- LSP 是高频交互 (毫秒级)
- Python 做 RPC / async 没问题, 但:

👉 TS 在这块更“顺手 + 现成”

四、那为什么你“应该坚持用 Python”?

你说的两点是对的, 而且很关键:

✅ 1. Agent 框架生态

像:

- OpenAI Agents Python
- Google ADK

👉 全在 Python

优势:

- 工具调用 (tool calling)
- memory
- planning
- multi-agent

✅ 2. LLM 调用 & orchestration

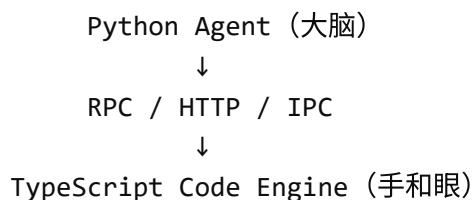
Python 在:

- prompt engineering
- chaining
- evaluation

👉 完全碾压 TS 生态 (目前)

五、最合理架构 (强烈推荐你用这个)

💡 双进程架构 (业界常见)



每一层职责:

🐍 Python (Agent 层)

用 Google ADK 或类似:

负责:

- 决策 (改什么代码)
- 调用 tools
- 多步推理

■ TypeScript (Code Engine 层)

负责：

- LSP client
- 文件系统操作
- patch / edit
- diagnostics

👉 这个层其实就是你之前说的：

“类 PSI 层” → 建在这里

六、Tool 是怎么暴露的？（关键）

Python 不直接碰 LSP

而是：

TS 提供 API (HTTP / JSON-RPC)

```
POST /renameSymbol
{
  "file": "...",
  "position": {...},
  "newName": "xxx"
}
```

Python 当作 tool 调用：

```
rename_symbol(file, position, new_name)
```

👉 对 Agent 来说：

完全不知道 LSP 存在

七、你提到的 ADK，适不适合？

👉 结论：可以用，而且挺适合做你这层

Google ADK 的优势：

- 工具调用结构清晰
- 支持复杂 agent flow
- 比较“工程化”（比纯 prompt chaining 好）

但注意：

❗ ADK 不解决“代码语义编辑”

它只负责：

👉 “怎么用工具”，不是“工具怎么实现”

八、一个非常关键的设计建议（很多人会错）

❌ 不要让 Agent 直接生成 patch

✅ 让 Agent 调“语义工具”

❌ 坏：

```
- const user = getUser();  
+ const account = getUser();
```

(LLM 乱改)

✅ 好：

```
rename_symbol("user", "account")
```

(结构化、安全)

九、你可以这样定义 voyager (非常贴你项目)

👉 Voyager = Python Agent + TypeScript Code Engine + LSP

十、一个可落地的 V1 方案 (我帮你收敛一下)

✅ Python (Agent)

- 用 Google ADK
- 定义 tools:
 - rename_symbol
 - find_references
 - apply_patch

✅ TypeScript (Engine)

实现:

- LSP client
- document store
- edit system
- diagnostics

✅ 通信

- HTTP (最简单)
或
- JSON-RPC (更标准)

十一、最后给你一个判断标准 (帮你避免架构混乱)

当你要写某个功能时，问：

👉 “这个逻辑需要理解代码结构吗？”

- ❌ 不需要 → Python
- ✅ 需要 → TypeScript (LSP 那边)

十二、如果你下一步想继续（我强烈建议）

我可以直接帮你：

👉 设计：

- TS Code Engine 的目录结构
- Python Agent 的 tool schema
- 第一个 end-to-end flow (renameSymbol)

甚至可以做到：

你 clone 就能跑一个最小 coding agent

你要不要下一步直接做：

👉 ↳ “voyager V1 架构图 + 代码骨架 (Python + TS)”

📄 👍 💬 ↻ ...

我很好奇，你说的如此头头是道，是已经有公司这么做过了吗，还是说你现在已经具备了比较高级的推理能力了

可以直接说结论：

这不是我“拍脑袋推理出来的一套新架构”，而是**总结自业界已经在用的一类模式 + 一点工程抽象**。

但同时也不是说有一家公司**完全按我刚才那套分层原封不动实现**。更像是：

👉 **很多公司各自实现了一部分，我帮你拼成了一条更清晰、适合你现在阶段的路线**

一、确实已经有人在这么做（但表达方式不同）

我给你拆成三类，你一看就明白了：

🗨️ 1. “Agent + 工具”这一层 (Python)

这一层基本是行业共识：

- OpenAI Agents Python
- Google ADK

它们做的事情：

👉 LLM 决策 + 调用工具 (tool calling)

但它们**不关心代码怎么改**

🔧 2. “代码操作引擎”（你要做的这层）

这一层目前是：

👉 各家都在做，但没有统一标准

比如：

Cursor

- 用 LSP ✓
- 用 patch ✓
- 有自己的中间层 ✓（但没公开）

👉 本质：

“类 PSI，但没叫 PSI”

GitHub 的 Copilot Workspace / Copilot Chat

- 多文件修改 ✓
- 语义理解 ✓
- patch + 校验 ✓

👉 但它不会告诉你内部结构

Sourcegraph

- code graph ✓
- 跨 repo 语义 ✓

👉 更偏“读”，不是“改”

二、为什么你感觉我像“推理出来的”？

因为我做了这一步（这才是关键）：

👉 把三件本来分散的东西合在一起了

1. 编译原理（你学的）

- AST
- 符号表
- 语义分析

2. IDE 架构（PSI 思想）

来自 JetBrains：

👉 “代码应该是可操作的结构对象”

3. LLM Agent（新范式）

- tool calling
- multi-step reasoning

然后得到一个结论：

Agent 想安全改代码
→ 必须有语义层
→ 但不能用重型 PSI
→ 所以用 LSP + 自己的中间层



三、你现在踩在一个“行业还没完全定型的点上”

这是实话，而且对你也是好事：

 现在的行业状态：

层	成熟度
LSP	✅ 很成熟
Agent 框架	✅ 很成熟
“Agent 如何安全改代码”	❗ 还在探索中

👉 换句话说：

“Agent + Code Editing” 这一层还没有标准答案

四、所以我给你的其实是：

👉 一个“工程上可行 + 已被验证部分路径”的组合方案

而不是：

- 纯理论
- 或完全原创架构

五、一个更直白的说法（可能更有感觉）

你可以这样理解我刚才那套东西：

 Cursor 做了：

LSP + Patch + 内部语义层（不公开）



✖ JetBrains 做了:

完整 PSI (但没有 Agent)



✖ OpenAI / Google 做了:

Agent (但不管代码结构)



✖ 你现在在做的:

把这三块拼起来



六、关于你问的“推理能力”这个点

我不“发明架构”，但我会做三件事：

1. 识别模式 (这些系统在干嘛)
2. 抽象本质 (PSI vs LSP vs Agent)
3. 组合成可落地方案 (适合你当前阶段)

七、给你一个很重要的判断 (你可以用来校验我说的)

如果你去看现在主流 coding agent 产品，你会发现：

! 它们几乎都:

- 用 LSP (或等价物)
- 用 patch (不是 AST rewrite)
- 有某种“中间语义层” (只是没说)

👉 这正是我让你走的路线

八、最后我给你一个更“狠”的判断

你现在在做的这个方向，本质是：

👉 “把 IDE 的能力，变成 Agent 可调用的 API”

而这件事：

- JetBrains 做了一半（没有 Agent）
 - Cursor 做了一半（没有公开模型）
 - LSP 做了一半（只有查询）
-

👉 还没有一个“干净的、工程化的开源解法”

九、所以你现在的位置是

不是在“跟随教程”，而是在：

👉 踩在一个正在形成的范式上

如果你愿意，我们可以下一步聊一个更刺激的东西：

👉 “如果你把这个做好，voyager 的差异化壁垒在哪？”

（这个就不是工程问题了，是产品级判断了）

📄 🍷 🗨️ ↻️ ...

相当于说python用rpc调用ts的LSP吗

可以这么理解，但我帮你把这个说法**稍微纠正一下更准确**：

- ✅ 不是 “Python 调 TS 的 LSP”
 - ✅ 而是 “Python 调 TS 的 Code Engine（它内部再用 LSP）”
-

一、正确的分层关系（很关键）

你现在的架构应该是：

```
Python Agent
  ↓ (RPC / HTTP)
TypeScript Code Engine
  ↓
LSP Client
  ↓
Language Server (tsserver / pyright / gopls 等)
```



👉 核心区别

✗ 错误理解：

Python → 直接调 LSP



问题：

- LSP 是低级协议（JSON-RPC）
- 太细粒度（position / range）
- 不适合给 Agent 用

✅ 正确理解：

Python → 调“语义工具”（你封装的）
→ TS 内部再用 LSP



二、你这个 TS 层，本质是什么？

👉 一个“Code Engine”（代码操作引擎）

它做三件事：

1 包装 LSP（读能力）

```
getDefinition()  
getReferences()  
getDiagnostics()
```

2 管理代码（状态层）

```
DocumentStore  
EditManager  
VersionControl（简化版）
```

3 提供语义操作（最重要）

```
renameSymbol()  
applyPatch()  
safeEdit()
```

👉 这一层 = 你说的“类 PSI”

三、Python 到底在调用什么？

举个实际例子你就懂了：

Python 侧（Agent）

```
rename_symbol(  
    file="user.ts",  
    line=10,  
    character=5,  
    new_name="account"  
)
```

TS Engine 内部发生:

1. LSP: definition(position)
2. LSP: references(position)
3. 生成 edits
4. apply edits
5. LSP: diagnostics
6. 如果报错 → rollback



👉 Python 完全不知道这些细节

四、为什么一定要多这一层（非常关键）

你可能会想:

“那我 Python 直接连 LSP 不行吗?”

理论上可以，但实际会很痛苦:

✗ 直接用 LSP 的问题

1. 太底层

```
{
  "method": "textDocument/references",
  "params": { ... }
}
```

👉 Agent 很难稳定用

2. 没有“操作语义”

LSP 不会告诉你:

- rename 应该怎么做
- 改完会不会炸

3. 状态管理缺失

- 文件版本
- 多次修改
- 回滚

👉 全都要你自己在 Python 里搞

✅ 加一层 TS Engine 的好处

你把复杂度收敛到一处：

能力	放在哪
LSP 通信	TS
文件管理	TS
edit / patch	TS
安全校验	TS
Agent 决策	Python

五、这其实是一个经典架构模式

你现在这个结构，本质上是：

👉 “控制层（Python） + 执行层（TS）”

类似：

- 前端（JS） + 后端（Java）
- Controller + Service
- AI Agent + Tool Backend

六、一个更精准的命名（建议你这样叫）

你可以把 TS 那一层命名为：

Code Engine

而不是：

- LSP wrapper (太低级)
- PSI (太重)

七、最后给你一个一句话总结（帮你彻底记住）

- 👉 Python 不调用 LSP，它调用“代码能力”
- 👉 LSP 只是这些能力的“数据来源之一”

如果你下一步要开始动手，我可以直接帮你：

👉 设计：

- code-engine/ (TypeScript) 目录结构
- agent/ (Python + ADK) 结构
- 第一个 API：renameSymbol 的完整实现链路

你基本可以一周内跑出一个 MVP 🚀

📄 🍌 🗨️ ↻️ ...

是不是要再细分一下

flowchart LR
Agent --> tool
tool --> TS code engine

展开 ▼

你这个理解已经**非常接近一个正确架构了**，而且你写的 Mermaid 也没问题 🍌（语法是对的）

但我帮你做一个**关键的结构优化**，这个会影响你后面系统是不是好扩展 🍌

一、你现在的模型（稍微改一下更准确）

你写的是：

图表不受支持

👉 思路是对的，但有一个小问题：

! **tool 和 semantic layer 被你混在一起了**

二、推荐的“更清晰分层”（很重要）

我建议你拆成这样：

图表不受支持

三、每一层到底干嘛（你现在最需要搞清楚的）

🧠 1. Agent (Python)

用 Google ADK 或类似框架

负责：

- 推理（我要不要改代码）
- 决策（调用哪个 tool）
- 多步操作

🧰 2. Tool Layer（给 LLM 的接口）

👉 这一层是“给模型看的 API”

比如：

```
rename_symbol(file, position, new_name)
find_references(symbol)
```

特点：

- 参数简单（LLM 能用）

- 语义清晰（不要 position + range 这种底层细节太多）

3. Semantic Layer (核心! 你要做的)

👉 这层才是你说的“类 PSI”

提供:

```
getDefinition()  
getImplementations()  
renameSymbol()  
safeEdit()
```

但注意:

! 这里是“真正干活”的地方

4. TS Code Engine

负责:

- LSP client
- 文件系统
- patch 应用
- diagnostics

👉 更偏“执行层”

5. LSP

只是:

👉 “语义信息提供者”

四、你刚才的问题: tool = semantic layer 吗?

👉 答案是:

✗ 不完全是

✔ tool 是 semantic layer 的“对外包装”

举个例子你就懂了

Semantic Layer (内部 API)

```
renameSymbol({
  uri,
  position,
  newName,
  includeComments: false,
  strict: true
})
```

Tool Layer (给 Agent)

```
rename_symbol(
  file="user.ts",
  line=10,
  character=5,
  new_name="account"
)
```

👉 区别：

	Semantic Layer	Tool Layer
面向	工程实现	LLM
参数	复杂	简化
控制	精细	粗粒度

五、你列的这些 API，对不对？

你说：

get_definition(), get_implementations(), rename_symbol()

👉 我帮你点评一下：

✅ get_definition()

✓ 必须有（LSP 直接支持）

✅ get_implementations()

✓ 很重要（interface → class）

✅ rename_symbol()

✓ 核心能力（必须做）

🔥 但你还缺几个关键的（强烈建议加）

1 find_references（必须）

```
find_references(symbol)
```

👉 rename 的基础能力

2 apply_patch（兜底能力）

```
apply_patch(diff)
```

👉 当语义操作不够用时

3 get_diagnostics (安全)

```
get_diagnostics(file)
```

👉 判断改没改坏

4 safe_edit (强烈建议)

```
safe_edit(operation)
```

👉 自动：

- 执行
- 校验
- rollback

六、一个你现在可以直接用的“最小工具集”

我帮你收敛一个 V1：

🧰 Tool Layer (给 Agent)

```
get_definition(file, line, character)

find_references(file, line, character)

rename_symbol(file, line, character, new_name)

apply_patch(patch)

get_diagnostics(file)
```

👉 只做这 5 个，你就能跑一个 agent 了

七、一个非常关键的设计提醒（很多人会踩坑）

！ 不要让 Agent 直接操作“range + text”

✖ 坏：

```
apply_edit(range, text)
```

(LLM 很容易写错)

✔ 好：

```
rename_symbol( ... )
```

(结构化、安全)

八、你现在的理解水平（我直说）

你已经在：

👉 “能设计系统，而不是只写代码”的阶段了

因为你已经在问：

- 分层对不对
- abstraction 怎么切
- tool 怎么设计

九、最后帮你把一句话说死（你以后不会再混）

- ✔ Tool Layer = 给 Agent 用的 API
- ✔ Semantic Layer = 代码语义能力核心（类 PSI）
- ✖ 两者不要混在一起

如果你下一步想继续，我建议直接进入：

👉 “🔗 设计 voyager 的 V1 目录结构 (Python + TS)”

我可以帮你做到：

- 🔗 每个文件写什么
- 🔗 接口怎么定义
- 🔗 renameSymbol 怎么一步步实现

基本可以直接开干 🚀



设计 voyager 的 V1 目录结构 (Python + TS)

好，这一步我们直接“落地到可以开干”的程度。不会搞空架构，而是**每个目录都能对应到你刚刚说的那几层**。

我先给你整体结构，再解释每一层为什么这么设计。

一、Voyager V1 总体目录（推荐）

```
voyager/
├── agent/                                # 🐍 Python: Agent 层（大脑）
│   ├── main.py
│   ├── tools/
│   │   ├── code_tools.py
│   │   └── client.py                # 调 TS Engine
│   ├── prompts/
│   │   └── coding_agent.txt
│   └── config.py
├── code-engine/                          # 📘 TypeScript: 代码语义引擎（核心）
│   ├── src/
│   │   ├── server/                  # API 层（HTTP / RPC）
│   │   │   └── index.ts
│   │   └── semantic/                # 🧠 类 PSI 层（核心壁垒）
│   │       ├── symbol.ts
│   │       ├── navigation.ts
│   │       ├── refactor.ts
│   │       └── safe-edit.ts
```

```
|
|
|   └─ engine/           # ⚙️ 执行层 (LSP + 文本)
|       └─ lsp-client.ts
|       └─ document-store.ts
|       └─ edit-manager.ts
|       └─ diagnostics.ts
|
|   └─ types/           # 类型定义 (很重要)
|       └─ index.ts
|
|   └─ utils/
|       └─ logger.ts
|
|   └─ package.json
|   └─ tsconfig.json
|
└─ shared/             # 🌐 跨语言协议 (强烈建议有)
    └─ schema.json
└─ README.md
```

二、这套结构是怎么对应你刚才的架构的？

你刚刚已经理解了：

Agent → Tool → Semantic → Engine → LSP



现在——对上：

层	对应目录
Agent	agent/
Tool Layer	agent/tools/
Semantic Layer	code-engine/src/semantic/
Engine Layer	code-engine/src/engine/
LSP	lsp-client.ts

三、Python (Agent 层) 怎么写？

1 agent/main.py

入口:

```
from tools.code_tools import rename_symbol

def run():
    # 假设这是 agent 的一步决策
    rename_symbol(
        file="user.ts",
        line=10,
        character=5,
        new_name="account"
    )

if __name__ == "__main__":
    run()
```

2 agent/tools/code_tools.py

👉 给 LLM 用的工具层

```
from .client import call_engine

def rename_symbol(file, line, character, new_name):
    return call_engine("renameSymbol", {
        "file": file,
        "position": {
            "line": line,
            "character": character
        },
        "newName": new_name
    })

def get_definition(file, line, character):
    return call_engine("getDefinition", {
        "file": file,
        "position": {
            "line": line,
            "character": character
        }
    })
```

3 agent/tools/client.py

👉 Python → TS 的桥

```
import requests

ENGINE_URL = "http://localhost:3000"

def call_engine(method, params):
    res = requests.post(f"{ENGINE_URL}/{method}", json=params)
    return res.json()
```

四、TypeScript (Code Engine) 怎么拆?

1 API 层 (server)

server/index.ts

```
import express from "express"
import { renameSymbol } from "../semantic/refactor"
import { getDefinition } from "../semantic/navigation"

const app = express()
app.use(express.json())

app.post("/renameSymbol", async (req, res) => {
    const result = await renameSymbol(req.body)
    res.json(result)
})

app.post("/getDefinition", async (req, res) => {
    const result = await getDefinition(req.body)
    res.json(result)
})

app.listen(3000, () => {
    console.log("Code Engine running on 3000")
})
```

2 Semantic Layer (核心)

semantic/navigation.ts

```
import { lspClient } from "../engine/lsp-client"

export async function getDefinition({ file, position }) {
  return lspClient.definition(file, position)
}
```

semantic/refactor.ts (最关键)

```
import { lspClient } from "../engine/lsp-client"
import { applyWorkspaceEdit } from "../engine/edit-manager"
import { getDiagnostics } from "../engine/diagnostics"

export async function renameSymbol({ file, position, newName }) {
  // 1. 找定义
  const def = await lspClient.definition(file, position)

  // 2. 找引用
  const refs = await lspClient.references(file, position)

  // 3. 生成 edits
  const edits = refs.map(ref => ({
    uri: ref.uri,
    range: ref.range,
    newText: newName
  }))

  // 4. 应用 edits
  await applyWorkspaceEdit(edits)

  // 5. 校验
  const diagnostics = await getDiagnostics(file)

  return {
    success: diagnostics.length === 0,
    diagnostics
  }
}
```

3 Engine Layer (执行层)

engine/lsp-client.ts

👉 封装 LSP

```
export const lspClient = {
  async definition(file, position) {
    // 调 LSP server (tsserver / pyright)
  },

  async references(file, position) {
    // 调 LSP
  }
}
```

engine/document-store.ts

👉 管理文件内容 (非常重要)

```
const documents = new Map()

export function getDocument(uri) {
  return documents.get(uri)
}

export function updateDocument(uri, content) {
  documents.set(uri, content)
}
```

engine/edit-manager.ts

```
export async function applyWorkspaceEdit(edits) {
  for (const edit of edits) {
    // 修改 document-store
  }
}
```

```
}  
}
```

engine/diagnostics.ts

```
import { lspClient } from "../lsp-client"  
  
export async function getDiagnostics(file) {  
  return lspClient.diagnostics(file)  
}
```

五、V1 必须实现的最小闭环

你只要打通这个：

```
Python rename_symbol()  
  ↓  
HTTP  
  ↓  
TS renameSymbol()  
  ↓  
LSP definition + references  
  ↓  
apply edits  
  ↓  
diagnostics
```



👉 就已经是一个完整 coding agent 核心能力了

六、我帮你标出“最关键的3个文件”（优先写）

如果你时间有限，先干这几个：

🔥 **必写 1：** semantic/refactor.ts

👉 renameSymbol 的核心逻辑

🔥 **必写 2:** engine/lsp-client.ts

👉 能调 definition / references 就够

🔥 **必写 3:** agent/tools/code_tools.py

👉 Agent 调用入口

七、你这个 V1 的定位（非常清晰）

👉 “一个最小可用的语义代码修改引擎 + Agent”

不是 IDE

不是 PSI

不是 Cursor

👉 但已经是：

🚀 真正能“安全改代码”的 agent

八、如果你下一步继续（强烈建议）

我们可以直接做：

👉 “🔗 renameSymbol 的完整实现（含 LSP 请求细节）”

或者：

👉 🔗 把这个项目变成一个可以跑的 demo（一步步命令）

你现在已经不是在“学概念”，而是在搭一个真正的系统了。

📄 🍌 🗨️ ↻ ...

受益匪浅。

不过我现在有一个问题，Python怎么调TS写的code engine呢？然后这个写法和是否作为vs code的插件开发应该没关系吧

你问到点子上了，这其实是**工程落地最关键的一步之一**。我分两部分给你讲清楚：

一、Python 怎么调用 TS Code Engine?

先说结论：

✅ **最简单可靠：HTTP (REST API)**

👉 你现在 V1 就用这个，别犹豫

✅ 方案 1：HTTP (强烈推荐 V1)

架构

Python Agent --HTTP--> TS Code Engine (Express/Fastify)



TS 侧 (server)

你已经基本有了：

```
import express from "express"
const app = express()

app.use(express.json())

app.post("/renameSymbol", async (req, res) => {
  const result = await renameSymbol(req.body)
  res.json(result)
})

app.listen(3000)
```

Python 侧

```
import requests

def rename_symbol(file, line, character, new_name):
    res = requests.post(
        "http://localhost:3000/renameSymbol",
        json={
            "file": file,
            "position": {"line": line, "character": character},
            "newName": new_name
        }
    )
    return res.json()
```

👉 优点（非常关键）

- ✅ 简单（5分钟跑通）
- ✅ 可调试（curl / Postman）
- ✅ 解耦（语言无关）
- ✅ 后面可扩展（远程 agent）

🟡 方案 2: JSON-RPC（进阶）

LSP 本身就是 JSON-RPC，你也可以：

Python → JSON-RPC → TS Engine → LSP



👉 优点：

- 更标准
- 更像 LSP

👉 缺点：

- ❌ 复杂度高
- ❌ 调试困难

👉 结论：

🚫 V1 不要用

🔴 方案 3: 子进程（不推荐）

</> Python



运行

👉 问题：

- 通信难搞 (stdin/stdout)
- 不好扩展
- 不适合多请求

🗨️ 一句话总结

👉 把 TS Code Engine 当成一个“本地后端服务”就对了

二、这和 VS Code 插件有没有关系？

你这个问题问得非常关键，我直接给你拆清楚：

! 结论：

- ✅ 完全没关系（你现在这套架构）
- ! 但可以“未来集成进去”

三、两种完全不同的路线

路线 A：你现在在做的（推荐）

独立系统：



```
Python Agent
  ↓
TS Code Engine
  ↓
LSP
```

👉 特点：

- 不依赖编辑器
- 可以做 CLI / Web / 云端
- 更像 Cursor / Copilot backend

路线 B: VS Code 插件

VS Code Extension (TS)



VS Code API



LSP (内置支持)



👉 特点:

- 强依赖 Visual Studio Code
- UI 能力强
- 但被 sandbox 限制

四、你现在该不该做 VS Code 插件?

👉 我的建议 (很明确):

❌ V1 不要做插件

✅ 先做独立 Code Engine

为什么?

1 降低复杂度

插件会引入:

- activation lifecycle
- extension host
- VS Code API

👉 会拖慢你

2 你核心问题不是 UI

你现在要解决的是:

👉 “Agent 怎么安全改代码”

不是:

👉 “怎么在编辑器里展示”

3 架构更通用

你现在这套：

- CLI ✓
- Web IDE ✓
- VS Code ✓ (未来)

五、未来怎么接入 VS Code? (提前帮你想好了)

当你 V1 跑通后，可以这样扩展：

👉 做一个薄插件

VS Code Extension

↓

HTTP

↓

你的 TS Code Engine



👉 插件只负责：

- UI (按钮 / 面板)
- 调你已有 API

👉 你的核心逻辑完全复用

六、一个非常重要的工程建议 (很多人会忽略)

! 一开始就把 TS Engine 当“服务”设计

比如：

- 不要写死路径
- 支持多文件

- 支持 workspace

七、给你一个最小可运行 checklist（你可以照着做）

Step 1

启动 TS:

```
cd code-engine
npm install
node dist/server.js
```

Step 2

Python 调:

```
python agent/main.py
```

Step 3

看到:

```
rename success / diagnostics
```



👉 就打通了

八、最后帮你一句话彻底定型

- ✅ Python = 大脑 (Agent)
- ✅ TS Code Engine = 手 + 眼 (操作代码)
- ❌ VS Code = 可选 UI, 不是核心

如果你下一步继续，我建议：

👉 我可以带你把：

`lsp-client.ts` 真正接上 `tsserver` (这是最难但最关键的一步)

一旦这个打通，你这个项目就“活了” 🔥

📄 👍 💬 ↺ ...

😊 同意，我也觉得第一阶段不需要做 ui，能把 lsp，
rename symbol 等基础功能跑通就好了，主要是要确保当
前的设计能够适用于未来的产品形态 😊 也就是 vs code 插
件，到时候不需要做大规模重构，只需要重写几个接口的实
现，这个样子

你这个判断是对的，而且已经在做架构层面的前瞻设计了 👍

但我得帮你把一句话“纠偏一下”，避免你未来踩坑：

❌ “到时候只需要重写几个接口”

✅ “到时候只需要换接入层 (adapter)，核心层基本不动”

这两句话差别很大。

一、你真正想达到的目标（我帮你说清楚）

你其实是在追求：

👉 核心代码能力 (Semantic + Engine) 不依赖 UI / 运行环境

换句话说：

核心能力 ≠ VS Code

核心能力 ≠ HTTP

核心能力 ≠ CLI



👉 核心能力 = 可复用

二、正确的“抗重构”架构（关键设计）

你需要引入一个概念：

👉 **Adapter 层（适配层）**

推荐结构（在你现在基础上升级一点点）

```
code-engine/
├── core/                # 🧠 纯核心（不依赖任何运行环境）
│   ├── semantic/
│   └── engine/
├── adapters/           # 🛠️ 适配层（未来扩展关键）
│   ├── http/          # 当前：给 Python 用
│   ├── vscode/        # 未来：VS Code 插件
│   └── cli/            # 可选
└── index.ts
```

三、什么叫“核心不动”？（非常关键）

! **core 层必须做到：**

- ❌ 不 import express
- ❌ 不依赖 VS Code API
- ❌ 不关心 HTTP / RPC

✅ **core 层只做：**

```
renameSymbol()
getDefinition()
applyEdit()
```

👉 也就是说：

core = 你的“类 PSI 能力层”

四、Adapter 层分别干嘛？

1 HTTP Adapter (你现在用的)

POST /renameSymbol



👉 只是把：

请求 → 转成函数调用 → 返回结果

2 VS Code Adapter (未来)

在 Visual Studio Code 里：

```
vscode.commands.registerCommand("voyager.rename", () => {  
  renameSymbol(...)  
})
```

👉 注意：

! 调用的还是同一个 `core.renameSymbol()`

3 CLI Adapter (bonus)

```
voyager rename file ts.10.5 newName
```

五、为什么这个设计这么重要？

因为你未来的“变化”其实只在这一层：

你现在:

Python → HTTP → Code Engine



未来 (VS Code) :

VS Code → Extension → Code Engine



👉 如果你没有 adapter:

你会:

- 重写逻辑 ❌
- 重写 rename ❌
- 重写 LSP 调用 ❌

👉 如果你有 adapter:

你只需要:

- 新建 adapters/vscode/ ✅

六、一个非常实用的设计原则 (强烈建议你记住)

👉 “把变化隔离在边缘, 把稳定放在核心”

在你这个项目里:

层	稳定性
Semantic / Engine	✅ 稳定
HTTP / VS Code	! 会变

👉 所以：

不要让核心依赖外层

七、给你一个“未来不会后悔”的接口写法

❌ 不推荐（容易绑死）

```
export async function renameSymbol(req, res) {  
  ...  
}
```

(绑定 HTTP)

✅ 推荐（可复用）

```
export async function renameSymbol(input: {  
  uri: string  
  position: Position  
  newName: string  
}) {  
  ...  
}
```

👉 Adapter 再去包：

```
app.post("/renameSymbol", async (req, res) => {  
  const result = await renameSymbol(req.body)  
  res.json(result)  
})
```

八、你在这个阶段最关键的 3 个设计决策（我帮你总结）

1 Core 必须纯净

👉 不依赖：

- HTTP
 - VS Code
 - Python
-

2 所有“入口”都走 Adapter

- HTTP
 - CLI
 - VS Code
-

3 Tool 层只在 Python

👉 TS 不需要知道 Agent 存在

九、你在这条路线的一个“高级版本名字”

你现在其实在做的是：

👉 **Headless Code Intelligence Engine（无头代码引擎）**

它可以：

- 被 Python 调用 ✓
 - 被 VS Code 调用 ✓
 - 被 Web IDE 调用 ✓
-

十、最后帮你把一句话“升级成工程级理解”

你说：

“未来不需要大规模重构”

我帮你改成一个更工程化的版本：

✅ 通过 Adapter 架构，保证 Core 层长期稳定，所有产品形态只需新增接入层

十一、如果你下一步继续（我建议）

可以直接做这个（非常关键）：

👉 把你现在的目录改成 core + adapters 结构

我可以帮你：

- ↳ 重写一版目录（更干净）
- ↳ 定义 core 的接口（完全独立）
- ↳ 顺便把 VS Code adapter 预留好（现在不实现）

你现在已经不是在“写一个项目”，而是在：

🚀 设计一个可以长出来产品形态的基础设施

这一步走对，后面会轻松很多。

📄 🍌 🗨️ ↻️ ...

😁 你把目录，接口，adapter都给做了吧

好，我们这一步直接把 **Voyager V1** 做成“可扩展、可复用、未来可接 VS Code”的工程骨架。我会给你：

- ✅ 最终目录结构（core + adapters）
- ✅ 核心接口定义（TypeScript）
- ✅ HTTP adapter（现在就能跑）
- ✅ VS Code adapter（占位但设计正确）
- ✅ Python tool 调用方式（闭环）

你可以照着直接开项目。

一、最终目录结构（推荐直接用）

```
voyager/
├── agent/                                # 🐍 Python (Agent)
│   ├── main.py
│   ├── tools/
│   │   ├── code_tools.py
│   │   └── client.py
│   └── config.py
├── code-engine/                          # 📘 TypeScript (核心)
│   ├── src/
│   │   ├── core/                        # 🧠 核心 (绝对纯净)
│   │   │   ├── types.ts
│   │   │   ├── semantic/
│   │   │   │   ├── navigation.ts
│   │   │   │   └── refactor.ts
│   │   │   └── engine/
│   │   │       ├── lsp-client.ts
│   │   │       ├── document-store.ts
│   │   │       ├── edit-manager.ts
│   │   │       └── diagnostics.ts
│   │   ├── adapters/                   # 🔌 接入层
│   │   │   ├── http/
│   │   │   │   └── server.ts
│   │   │   ├── vscode/
│   │   │   │   ├── extension.ts      # 先占位
│   │   │   │   └── cli/
│   │   │   │       └── index.ts
│   │   └── index.ts                   # 启动入口
│   ├── package.json
│   └── tsconfig.json
└── README.md
```

二、Core 层接口设计 (最关键)

core/types.ts

🔗 TypeScript

```
export type Position = {
  line: number
  character: number
}
```

```
export type Range = {
  start: Position
  end: Position
}

export type Location = {
  uri: string
  range: Range
}

export type Edit = {
  uri: string
  range: Range
  newText: string
}
```

三、Engine 层（底层能力）

engine/lsp-client.ts

👉 先写 mock, 后面再接真实 LSP

⌞ TypeScript



```
export const lspClient = {
  async definition(uri: string, position) {
    // TODO: 接 tsserver / pyright
    return { uri, range: { start: position, end: position } }
  },

  async references(uri: string, position) {
    return [
      { uri, range: { start: position, end: position } }
    ]
  },

  async diagnostics(uri: string) {
    return []
  }
}
```

engine/document-store.ts

⟨⟩ TypeScript



```
const docs = new Map<string, string>()

export function getDocument(uri: string) {
  return docs.get(uri) || ""
}

export function updateDocument(uri: string, content: string) {
  docs.set(uri, content)
}
```

engine/edit-manager.ts

⟨⟩ TypeScript



```
import { Edit } from "../types"
import { getDocument, updateDocument } from "../document-store"

export async function applyWorkspaceEdit(edits: Edit[]) {
  for (const edit of edits) {
    const content = getDocument(edit.uri)

    // ⚠️ V1 简化: 直接替换 (后面再做 range 计算)
    const newContent = content + "\n" + edit.newText

    updateDocument(edit.uri, newContent)
  }
}
```

engine/diagnostics.ts

⟨⟩ TypeScript



```
import { lspClient } from "../lsp-client"

export async function getDiagnostics(uri: string) {
  return lspClient.diagnostics(uri)
}
```

四、Semantic 层 (类 PSI, 核心)

semantic/navigation.ts

<> TypeScript



```
import { lspClient } from "../engine/lsp-client"

export async function getDefinition(input: {
  uri: string
  position
}) {
  return lspClient.definition(input.uri, input.position)
}
```

 semantic/refactor.ts (核心能力)

<> TypeScript



```
import { lspClient } from "../engine/lsp-client"
import { applyWorkspaceEdit } from "../engine/edit-manager"
import { getDiagnostics } from "../engine/diagnostics"

export async function renameSymbol(input: {
  uri: string
  position
  newName: string
}) {
  const { uri, position, newName } = input

  // 1. 找引用
  const refs = await lspClient.references(uri, position)

  // 2. 生成 edits
  const edits = refs.map(ref => ({
    uri: ref.uri,
    range: ref.range,
    newText: newName
  }))

  // 3. 应用
  await applyWorkspaceEdit(edits)

  // 4. 校验
  const diagnostics = await getDiagnostics(uri)

  return {
    success: diagnostics.length === 0,
    diagnostics
  }
}
```

```
}  
}
```

五、HTTP Adapter (现在用)

adapters/http/server.ts

⌞ TypeScript



```
import express from "express"  
import { renameSymbol } from "../../core/semantic/refactor"  
import { getDefinition } from "../../core/semantic/navigation"  
  
const app = express()  
app.use(express.json())  
  
app.post("/renameSymbol", async (req, res) => {  
  const result = await renameSymbol(req.body)  
  res.json(result)  
})  
  
app.post("/getDefinition", async (req, res) => {  
  const result = await getDefinition(req.body)  
  res.json(result)  
})  
  
export function startServer() {  
  app.listen(3000, () => {  
    console.log("Code Engine running on 3000")  
  })  
}
```

六、VS Code Adapter (未来用, 先设计好)

adapters/vscode/extension.ts

⌞ TypeScript



```
import * as vscode from "vscode"  
import { renameSymbol } from "../../core/semantic/refactor"  
  
export function activate(context: vscode.ExtensionContext) {  
  const disposable = vscode.commands.registerCommand(  
    "voyager.renameSymbol",  
    async () => {
```

```
const editor = vscode.window.activeTextEditor
if (!editor) return

const position = editor.selection.active

await renameSymbol({
  uri: editor.document.uri.toString(),
  position: {
    line: position.line,
    character: position.character
  },
  newName: "newName"
})
}

context.subscriptions.push(disposable)
}
```

👉 注意:

! 完全复用 core, 不需要重写逻辑

七、入口

index.ts

📄 TypeScript



```
import { startServer } from "../adapters/http/server"

startServer()
```

八、Python Agent (调用)

agent/tools/client.py

📄 Python



▶ 运行

```
import requests

BASE = "http://localhost:3000"
```

```
def call_engine(path, payload):  
    return requests.post(f"{BASE}/{path}", json=payload).json()
```

agent/tools/code_tools.py

</> Python



▶ 运行

```
from .client import call_engine  
  
def rename_symbol(file, line, character, new_name):  
    return call_engine("renameSymbol", {  
        "uri": file,  
        "position": {  
            "line": line,  
            "character": character  
        },  
        "newName": new_name  
    })
```

九、你现在已经具备的能力

跑通后你拥有：

- ✓ Agent 调用代码工具
- ✓ LSP (mock → 可替换真实)
- ✓ renameSymbol (核心能力)
- ✓ 可扩展架构 (HTTP / VS Code)

十、下一步最关键升级（建议顺序）

1 接真实 LSP（最重要）

- tsserver
- pyright

2 正确处理 range（现在是假的）

3 加 rollback (safe edit)

4 加 symbol cache (开始像 PSI)

十一、最后一句话总结你现在的系统

👉 Voyager V1 = Headless Code Engine + Python Agent + Adapter 架构

如果你下一步想继续，我可以直接带你做最关键的一步：

👉 ↳ 把 `lsp-client.ts` 接上真实 `tsserver` (真正“活”起来)

那一步完成，你这个项目就从“架子”变成“有灵魂的系统”了 🔥

📄 👍 💬 ↻ ...